

Exercise 4 Discussion

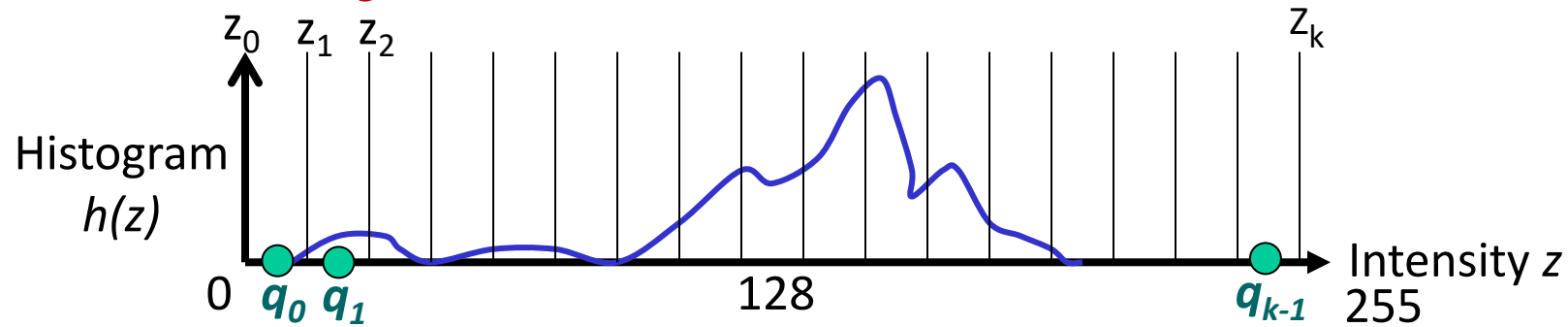
- Compute transformation - consecutive frames
- Compose transformations - a selected frame
- Warp frames (one option)
- Paste strips in output mosaics
- (2nd option) take a strip in output mosaic, find its location in the input frame, and warp+copy
- Each step should be debugged separately, and results saved.
- Good running time ~10 seconds (~10 input frames/second).

Efficiency Consideration

- Use Numpy vector operations
- Write to disk only results you need to save
- Compute alignment & warping only once
- Compute size of output images based on computed transformations.
- Presubmit: limited to 100 seconds.
- Presubmit: will check compile & Run on input sequence, giving desired number of views. Views will be checked manually by TAs.

Extra Compression

Grey Level Quantization



- Reduce number of grey levels
- Divide grey level range into fewer segments
- All grey level in each segment are mapped to one index
 - Borders of segments: $z_0=0, z_1, z_2, \dots, z_k=255$
 - All grey levels in segment $[z_{i-1}, z_i]$ are mapped to index $i - 1$
- Restoration: Each index i will be restored to intensity q_i
- Uniform Quantization: $z_{i+1} - z_i = (z_k - z_0)/k$ $q_i = (z_i + z_{i+1})/2$

Quantization Error

- Assume that during quantization grey level g is indexed by i , and index i is restored as q_i
- A Possible quantization error **for one pixel** with grey level g is:

$$E_g^2 = (g - q_i)^2$$

- The total error introduced by quantization of all pixels is:

$$E^2 = \sum_{\text{pixels } p} E_p^2 = \sum_{n=0}^{255} \text{hist}(n) E_n^2$$

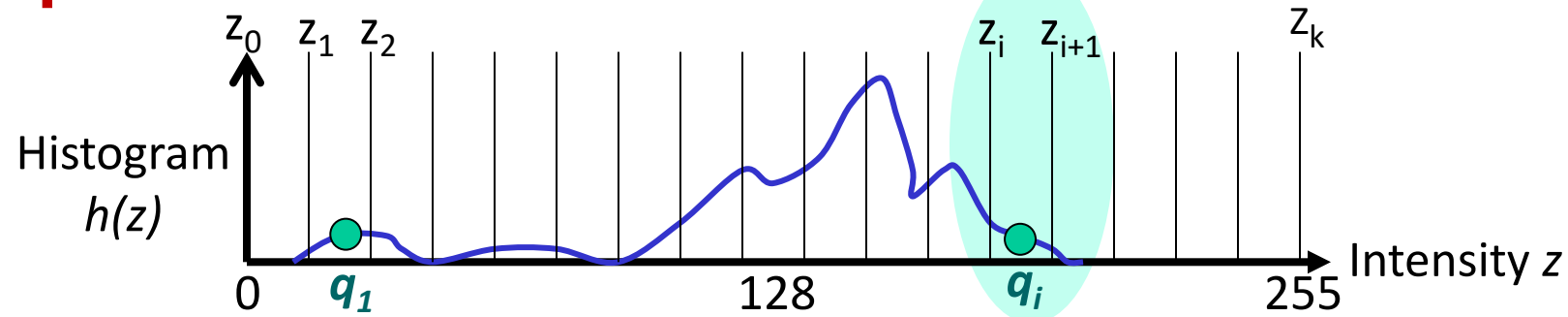
All pixels with same grey level will have same quantization error

- Error can be reduced by finding optimal transformations:
- $g \rightarrow i (z_i)$, $i \rightarrow q_i$

SSD As a Quantization Error

- SSD (L_2 , Sum of Squared Differences) is a poor error measure compared to human perception
- There are suggestions for other measures, but they are harder to compute and to analyze (E.g. L_1 , Sum of Absolute Values)
- $SSD = 0$ implies that we got the the same image...
- (Best way to compute image similarity is with Neural networks... Called “semantic distance”)

Optimal Quantization - Condition



- Minimize the error:
- Optimal Solution if: (**Prove!**)

$$\sum_{i=0}^{k-1} \left(\sum_{g=\lfloor z_i \rfloor + 1}^{\lfloor z_{i+1} \rfloor} (q_i - g)^2 h(g) \right)$$

$$q_i = \frac{\sum_{g=\lfloor z_i \rfloor + 1}^{\lfloor z_{i+1} \rfloor} g \cdot h(g)}{\sum_{g=\lfloor z_i \rfloor + 1}^{\lfloor z_{i+1} \rfloor} h(g)}$$

$$z_i = \frac{q_{i-1} + q_i}{2}$$

Weighted grey level average in segment

Optimal Quantization – EM (Expectation Minimization) (K-Means)

- Find z_i and q_i such that

$$q_i = \frac{\sum_{g=\lfloor z_i \rfloor+1}^{\lfloor z_{i+1} \rfloor} g \cdot h(g)}{\sum_{g=\lfloor z_i \rfloor+1}^{\lfloor z_{i+1} \rfloor} h(g)} \quad z_i = \frac{q_{i-1} + q_i}{2}$$

- This is done iteratively
- Initial guess: find z_i such that for all i there are same number of pixels whose grey level is between z_i and z_{i+1} .
- From initial guess of z_i compute q_i .
- Iterate: computing z_i and then q_i then z_i until convergence. 8

Compression Limitations

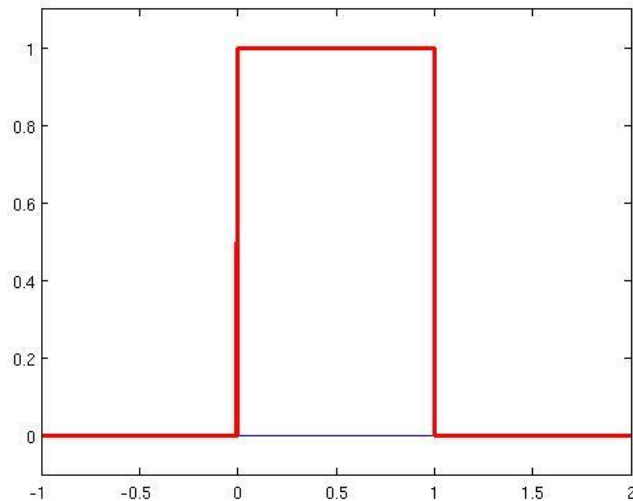
- JPEG is computed on 8x8 patches. Why 8x8?
- Fan-out issues: all pixels are accessed in Cosine Transform
- 64 pixels was, at the time, maximal number of memory connections for a chip (that will compute the cosine transform).
- Connection were needed for real-time JPEG in cameras.
- Can we address global transformations on one side, but with local computation (say 8x8) on the pother side?
- Pyramids! But pyramid compression increased the number of pixels by 33%

Compression with Wavelet Functions

1D Low-Pass / High Pass Orthogonal Pairs

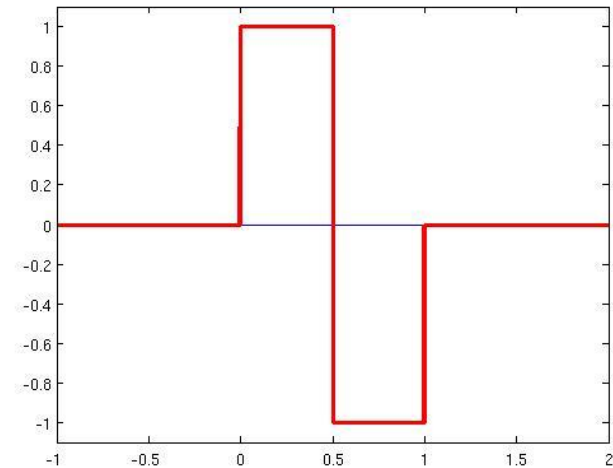
- In Pyramids, low-pass Gaussian and high-pass Laplacian are uniform in both X and Y directions. Need extra 33% memory.
- Wavelets are pairs of 1D low-pass/high-pass applied separately in each direction. Do not need any extra memory.

Low Pass (Scaling, Father) Filter



Sum of 1 : Convolution with $(1, 1)/2$

High Pass (Wavelet, Mother) Filter

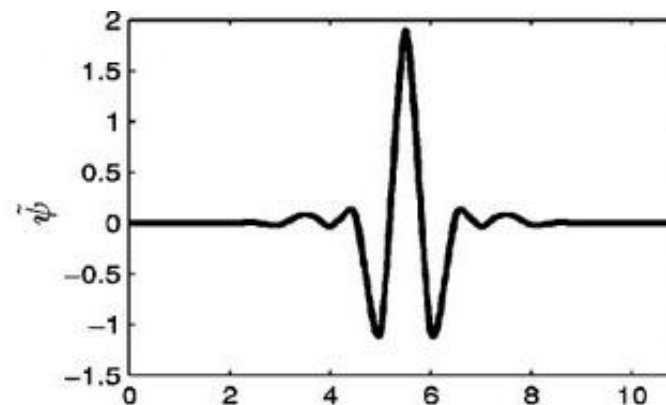
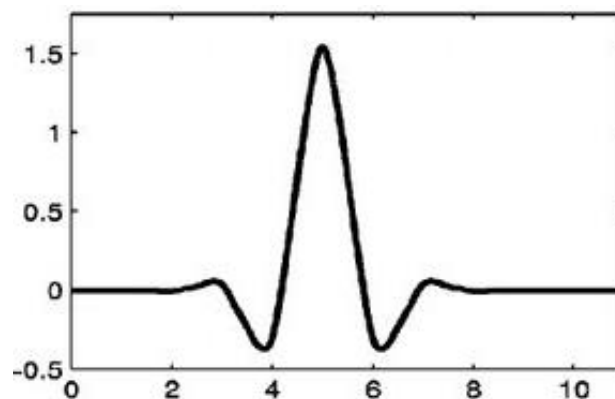


Sum of 0 : Convolution with $(1, -1)/2$

Haar Wavelets
Example
(Never used...)

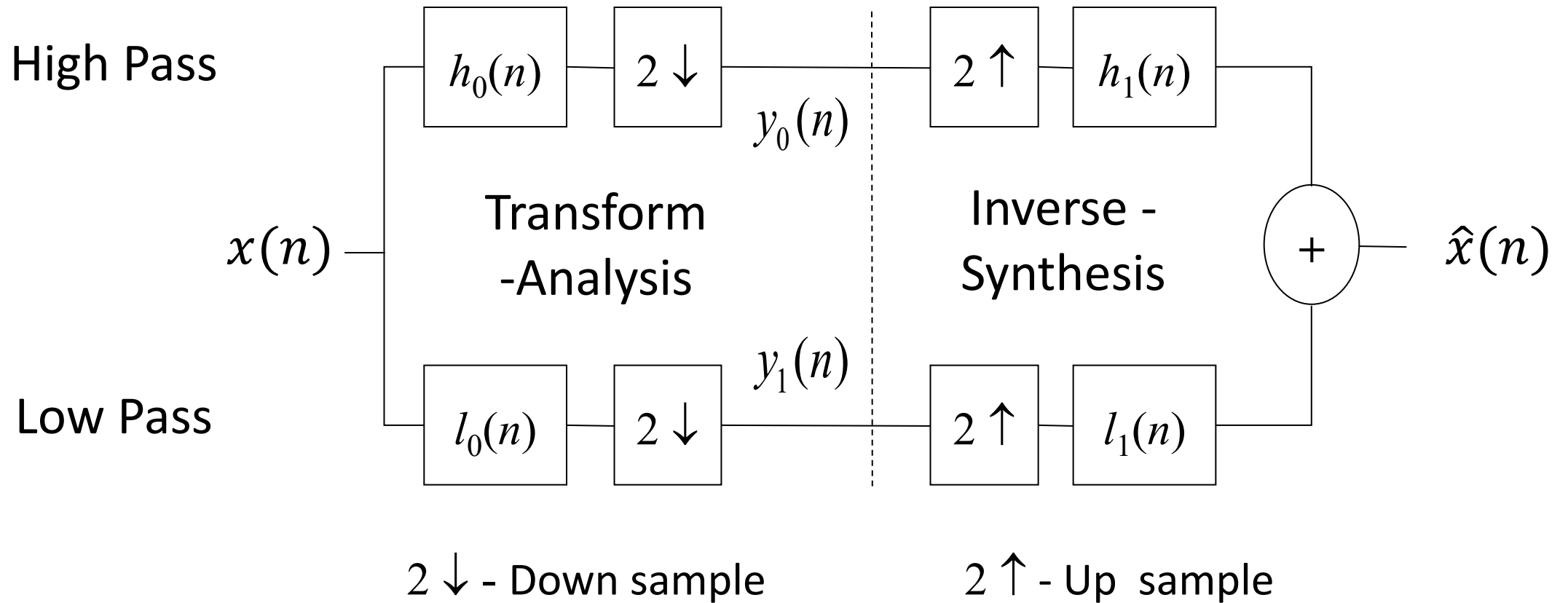
Daubechies 9/7-tap Wavelet Filter

Taps	Low Pass Filter ($\Sigma=1$)	Taps	High Pass Filter ($\Sigma=0$)
4	0.6029490183263579	3	1.1115087052456994
3, 5	0.2668641184428723	2, 4	-0.5912717631142470
2, 6	-0.07822326652898785	1, 5	-0.05754352622849957
1, 7	-0.01686411844287495	0, 6	0.09127176311424948
0, 8	0.02674875741080976		



Low (9-tap) = (0.02, -0.01, -0.07, 0.26, **0.60**, 0.26 -0.07, -0.01, 0.02) ($\Sigma=1$)
 High (7-tap) = (0.09, -0.05, -0.59, **1.11**, -0.59, -0.05, 0.09) ($\Sigma=0$)

Subband Coding (Wavelets)

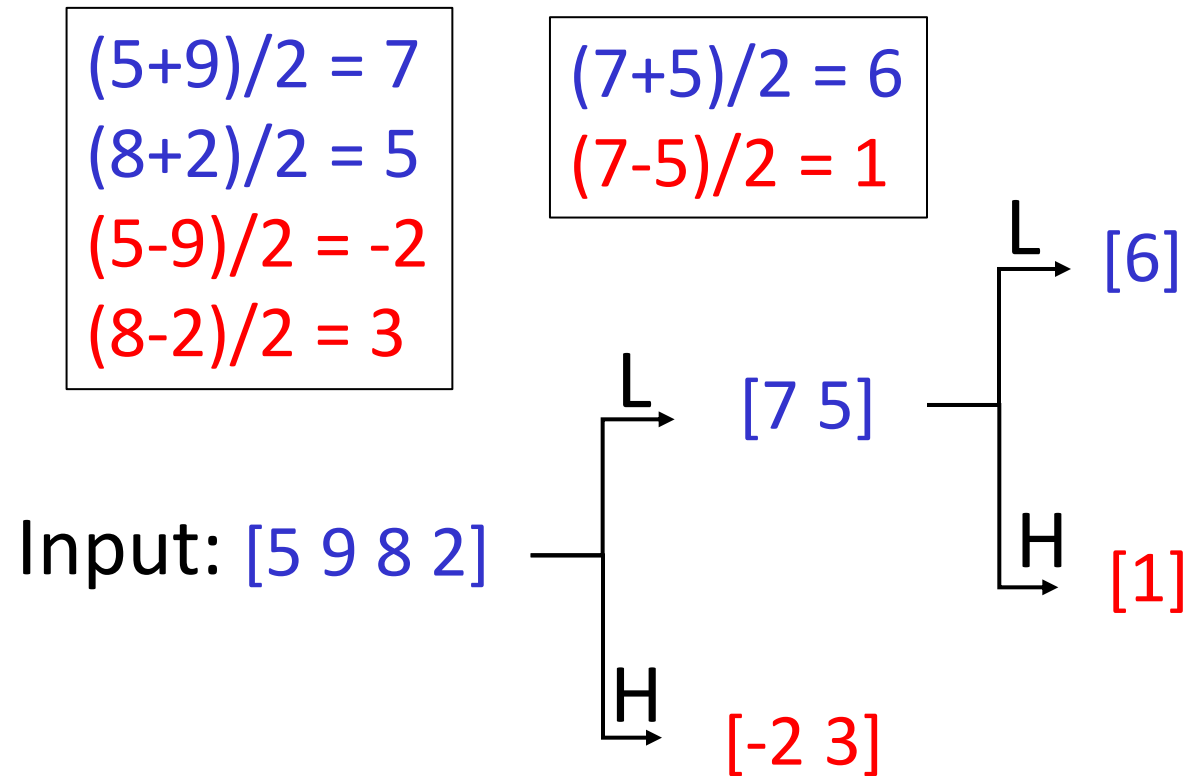


Intuition: Recover two numbers a and b from $\frac{a+b}{2}$ and $\frac{a-b}{2}$?

Recursive Splitting to Low Pass + High Pass (Haar)

$\frac{1}{2}[1 \ 1]$ – LP sampling

$\frac{1}{2}[1 \ -1]$ – HP sampling

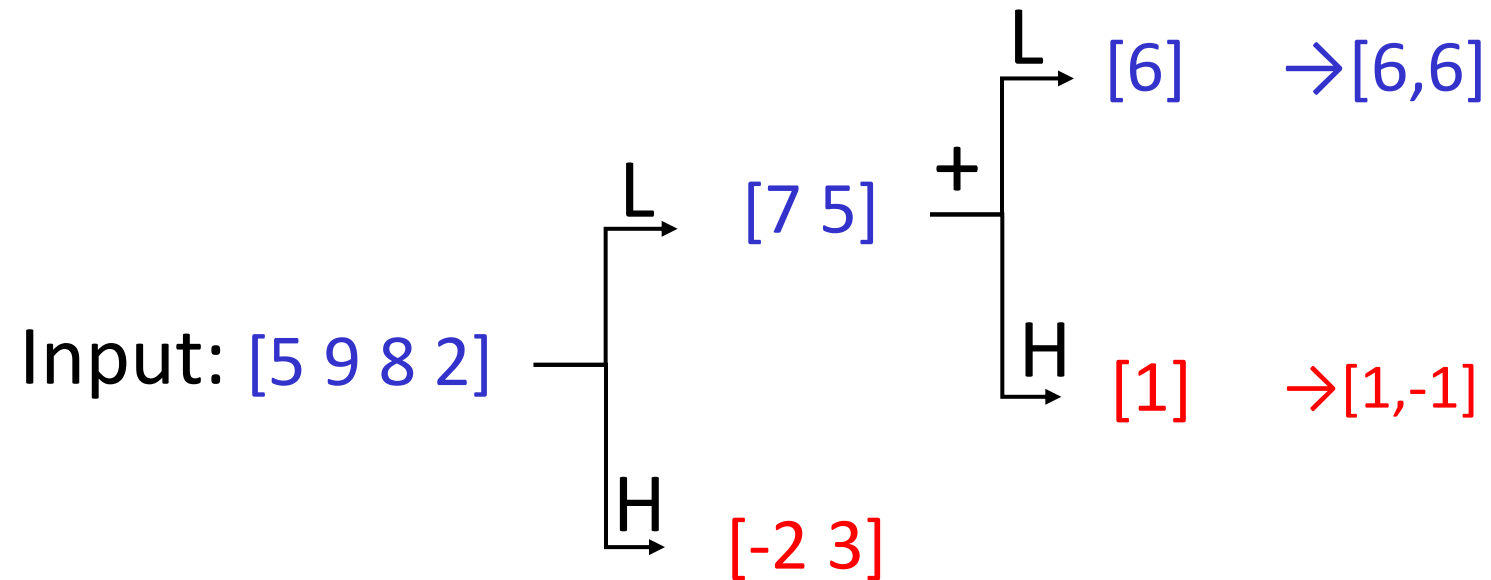


Transform result : [6 1 -2 3]

Reconstruction (Haar)

$\begin{bmatrix} 1 & 1 \end{bmatrix}$ – LP Up sampling

$\begin{bmatrix} 1 & -1 \end{bmatrix}$ – HP UP sampling

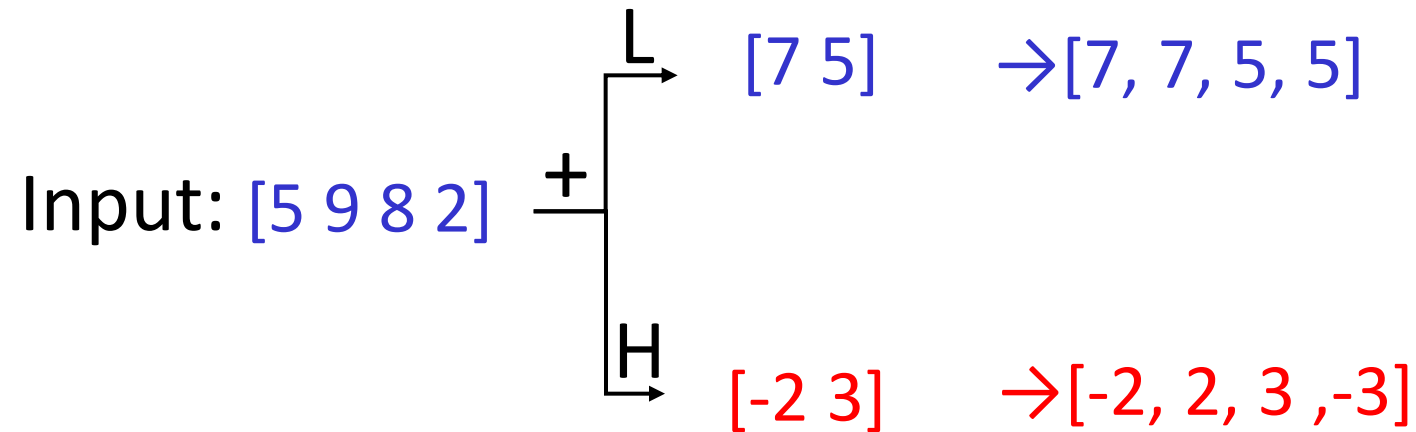


Transform result : $\begin{bmatrix} 6 & 1 & -2 & 3 \end{bmatrix}$

Reconstruction (Haar)

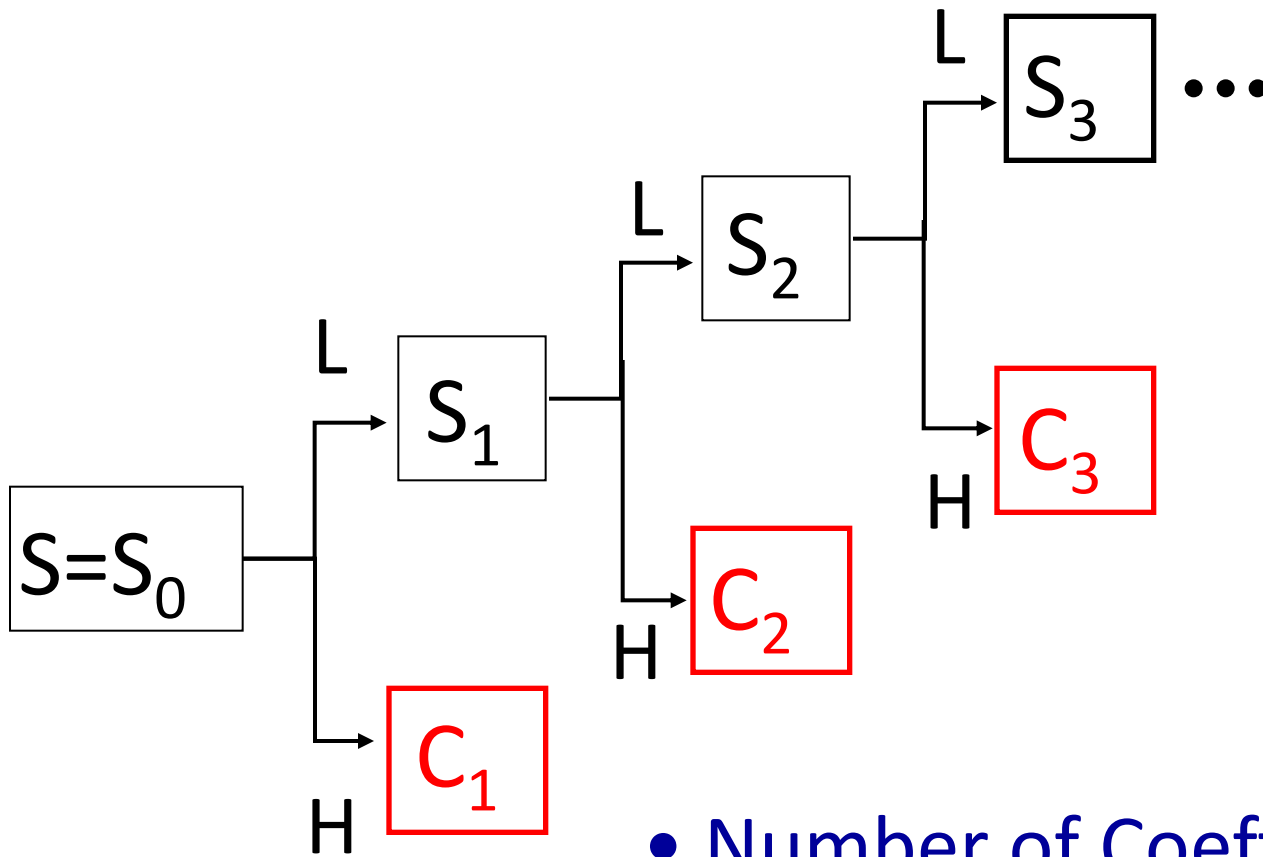
$\begin{bmatrix} 1 & 1 \end{bmatrix}$ – LP Up sampling

$\begin{bmatrix} 1 & -1 \end{bmatrix}$ – HP UP sampling



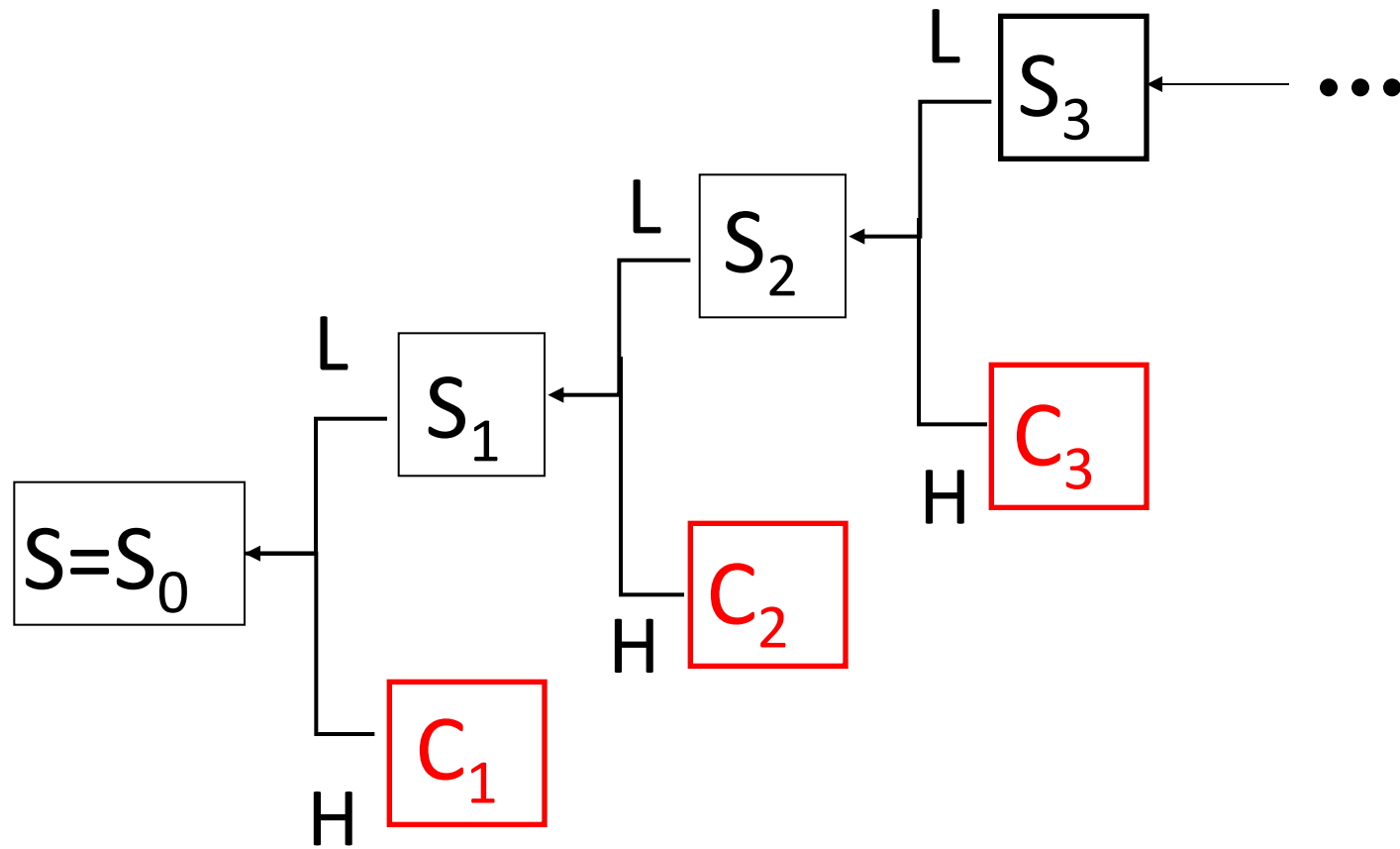
Transform result : $\begin{bmatrix} 6 & 1 & -2 & 3 \end{bmatrix}$

The Wavelet Pyramid – 1D

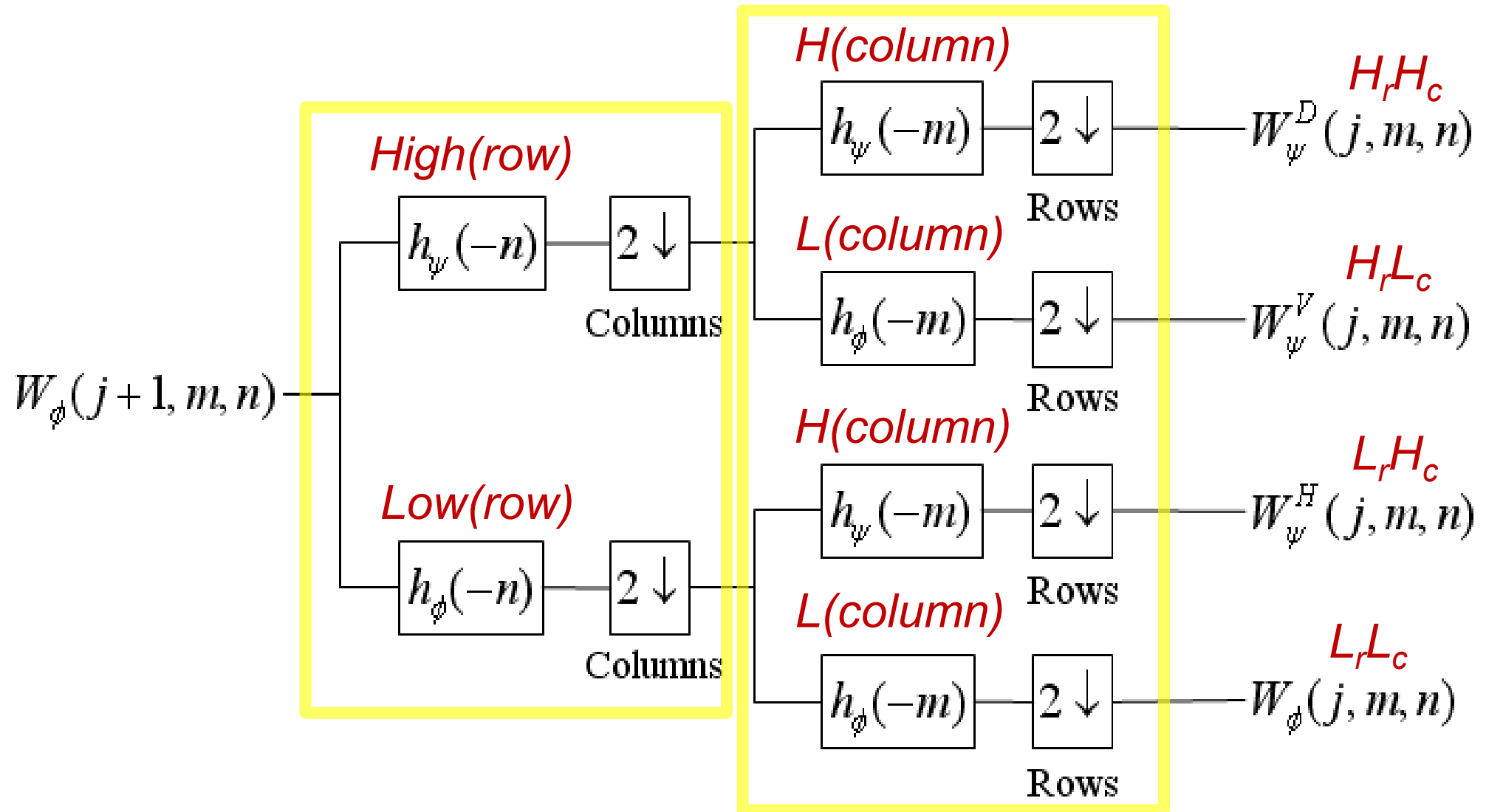


- Number of Coefficients is same as in original
- Entropy decreases in High-Pass components

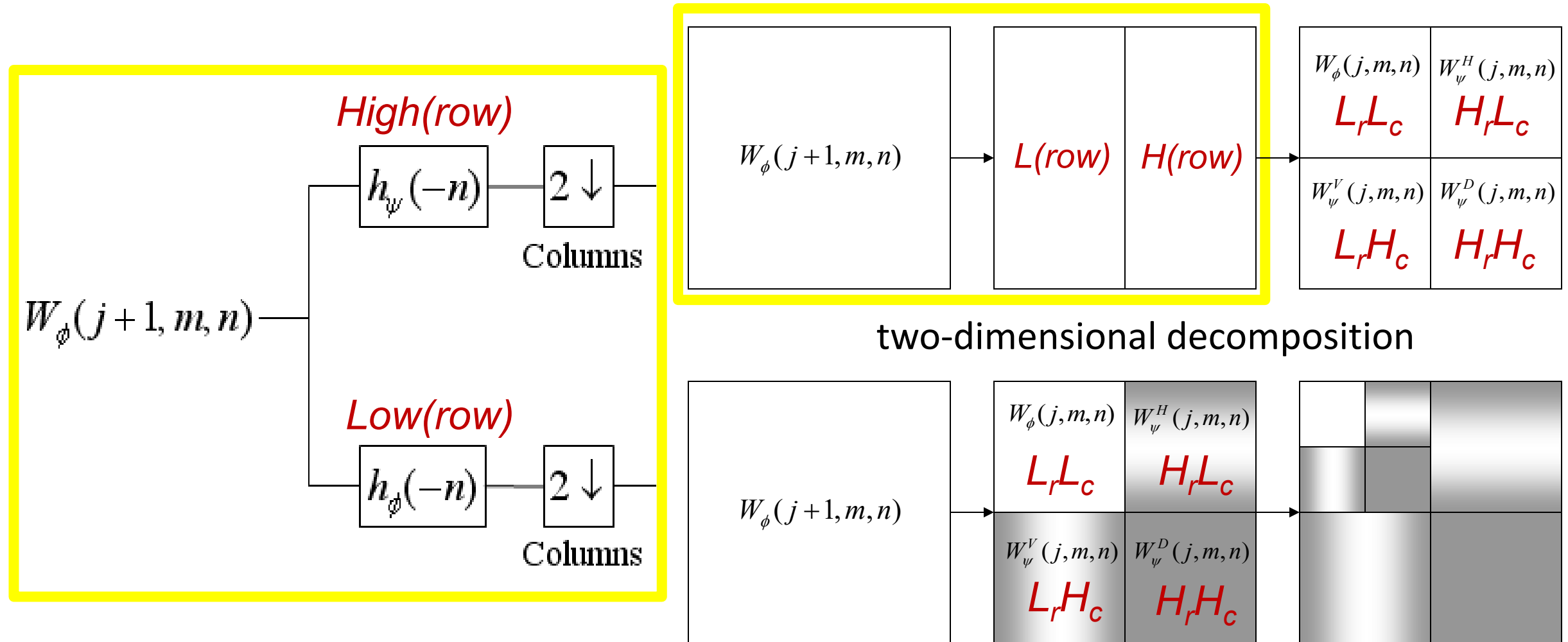
Reconstruction



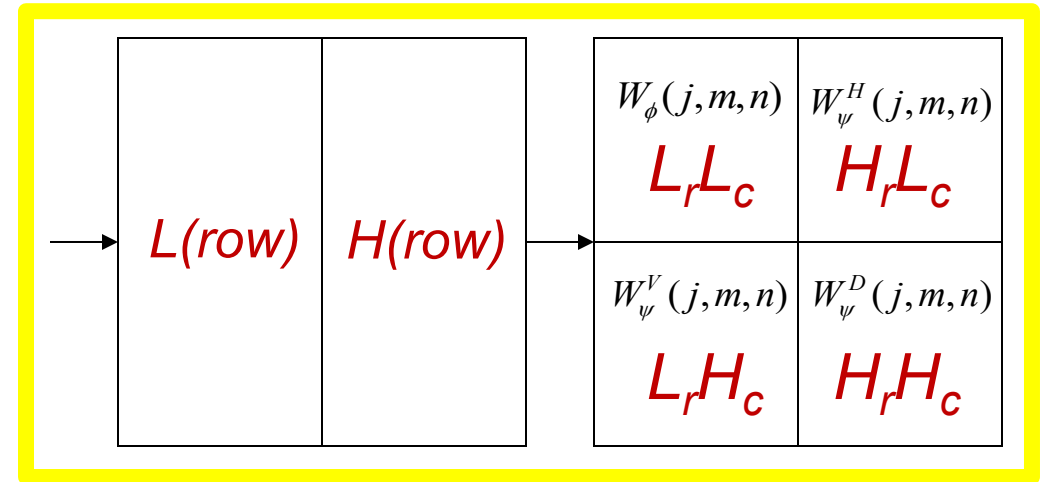
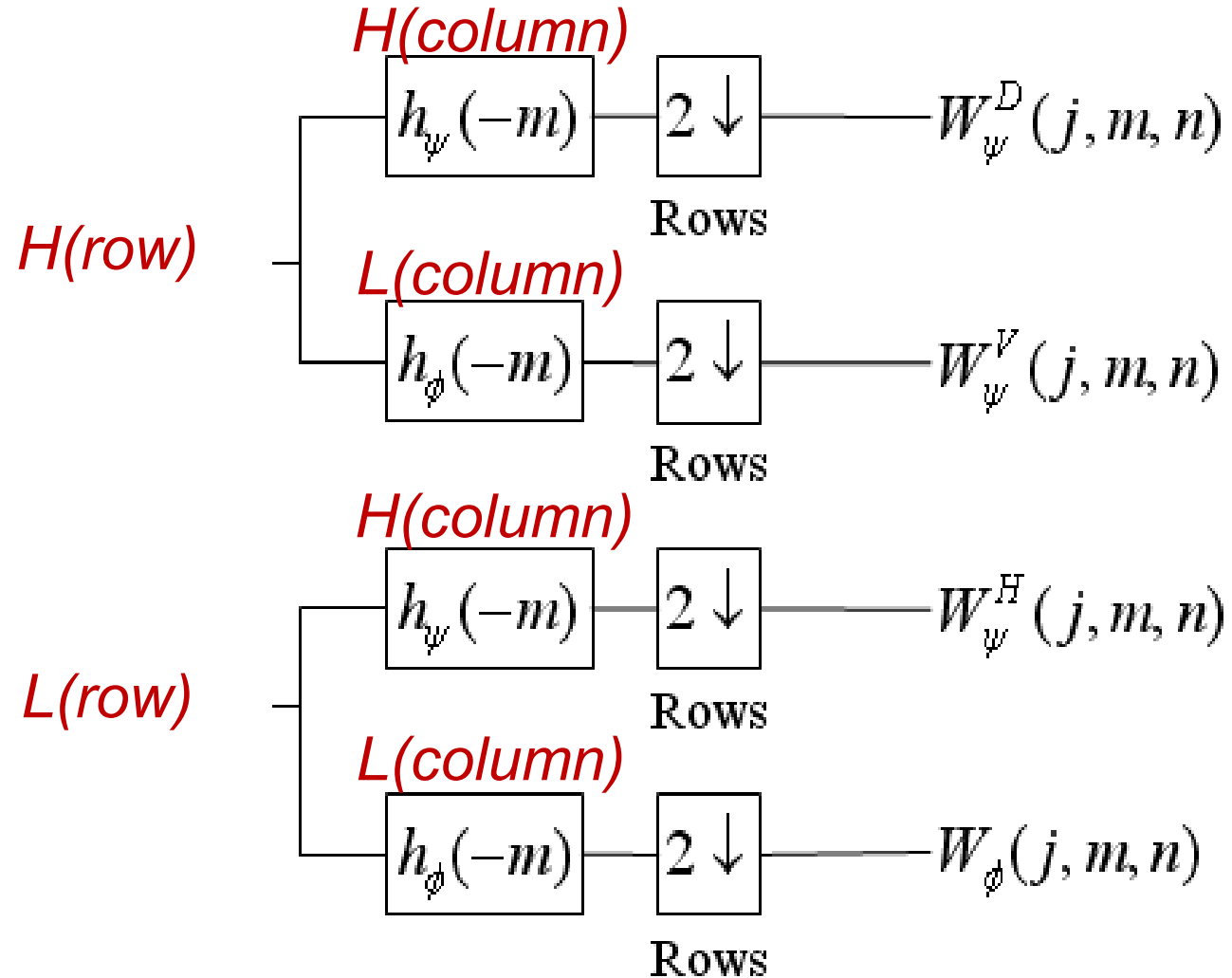
Wavelet Transforms in Two Dimensions



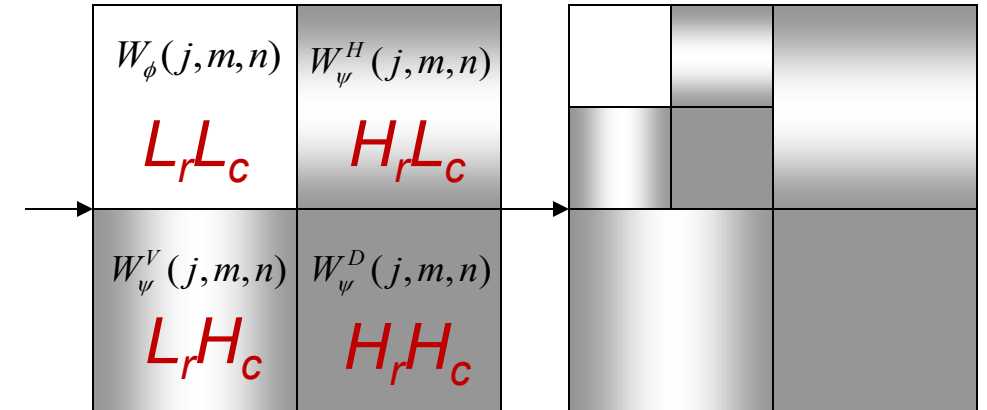
Wavelet Transforms in Two Dimensions



Wavelet Transforms in Two Dimensions



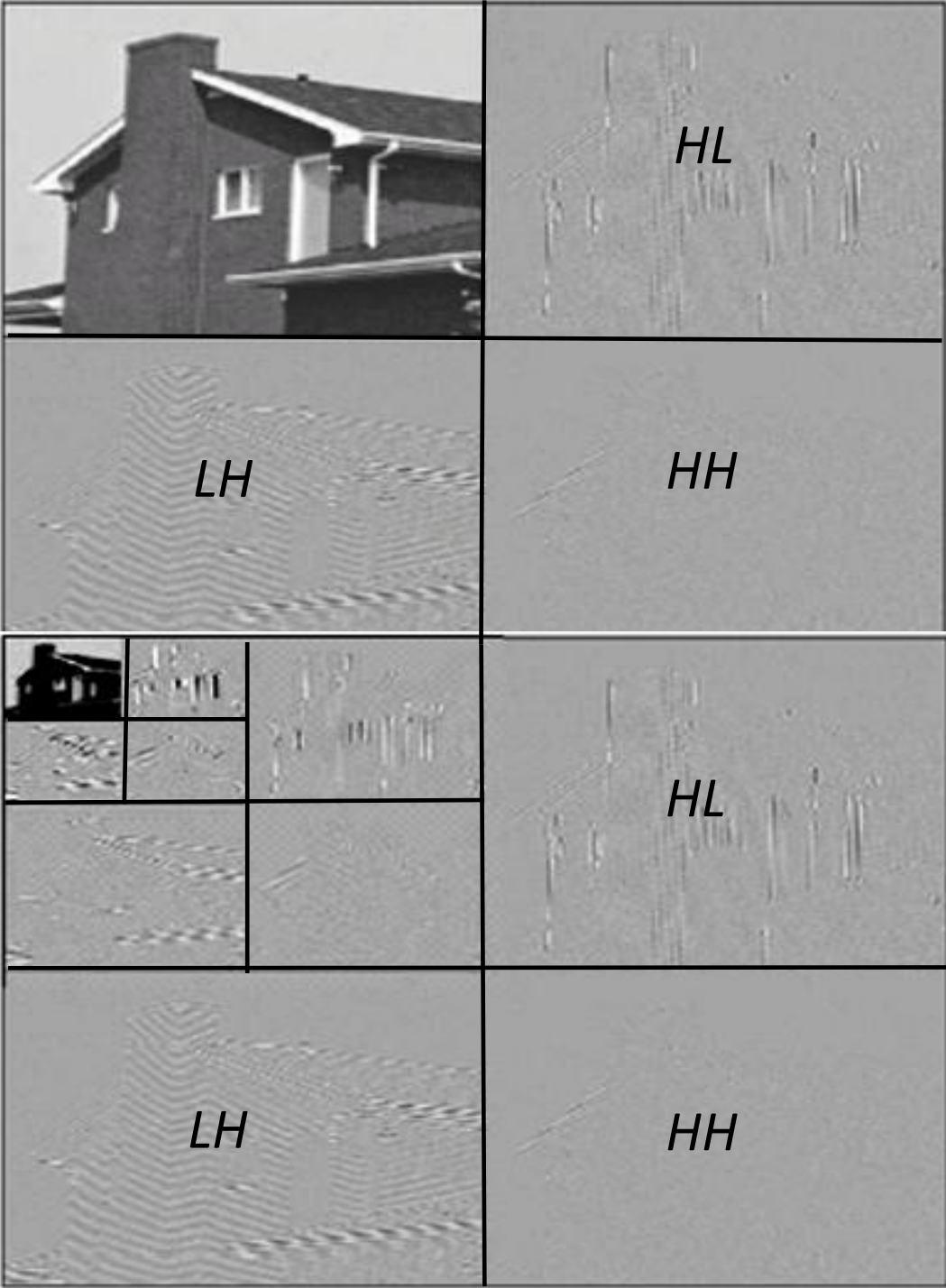
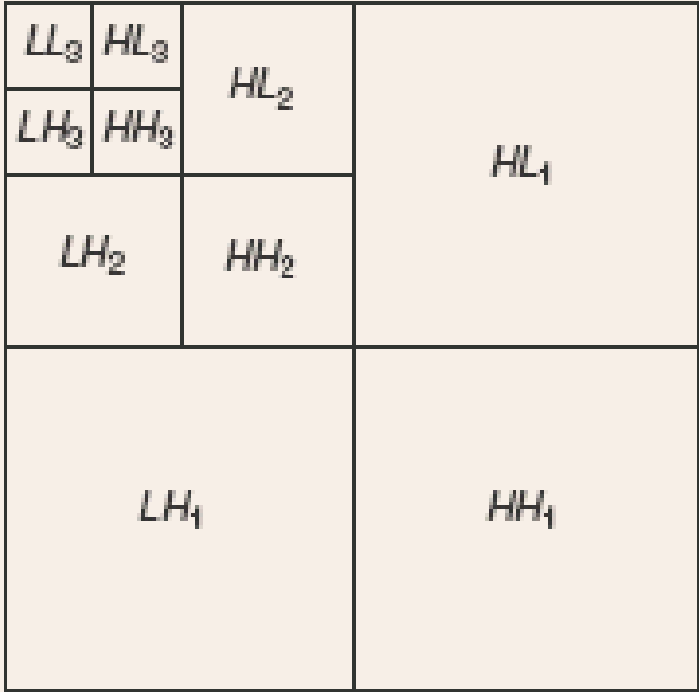
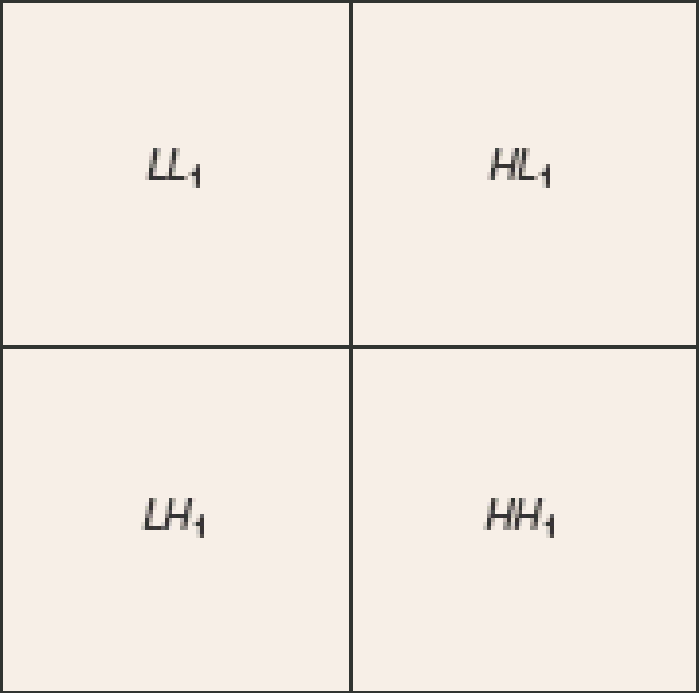
two-dimensional decomposition



Wavelet Decomposition Example

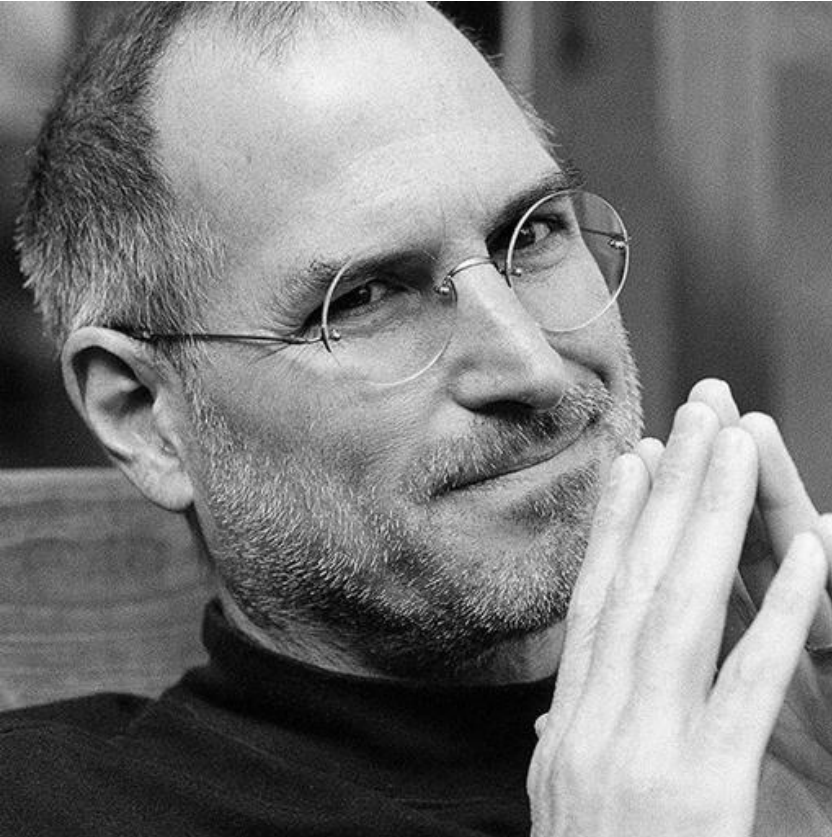


▲ 2. Original image used for demonstrating the 2-D wavelet transform.



Simulation Result of Wavelet Compression: RMS Error

Original image



DCT compression



Wavelet compression



CR = 1:51

RMS = 15.74

CR = 1:51

RMS = 13.68

CR = Compression Ratio

RMS = Root Mean Square

JPEG2000

vs.
JPEG

CR 1:200

JPEG



JPEG2000



CR 1:100



Video Compression

- Exploit *spatial redundancy* within frame
 - E.g. JPEG: transforming, quantizing, coding
- Exploit *temporal redundancy* between frames
 - Only the sun has changed position

Previous Frame

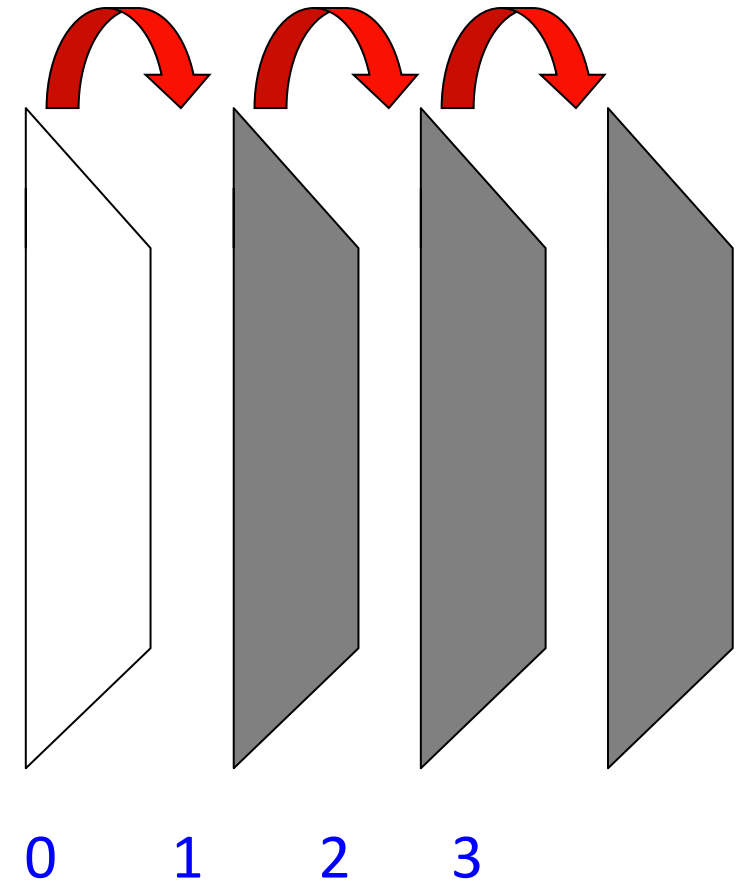


Current Frame



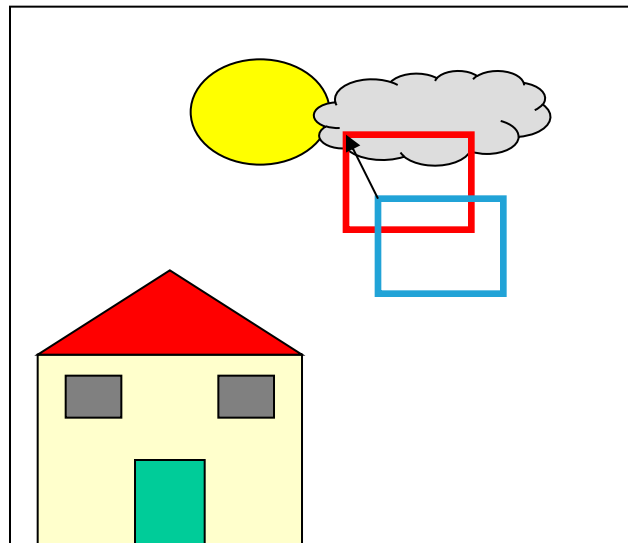
Simplest Temporal Coding

- Frame 0 (still image)
- Difference: Frame 1 – Frame 0
- Difference: Frame 2 – Frame 1
- If no movement, all differences are 0.
Can be compressed!
- If movement, can see it in the
difference images

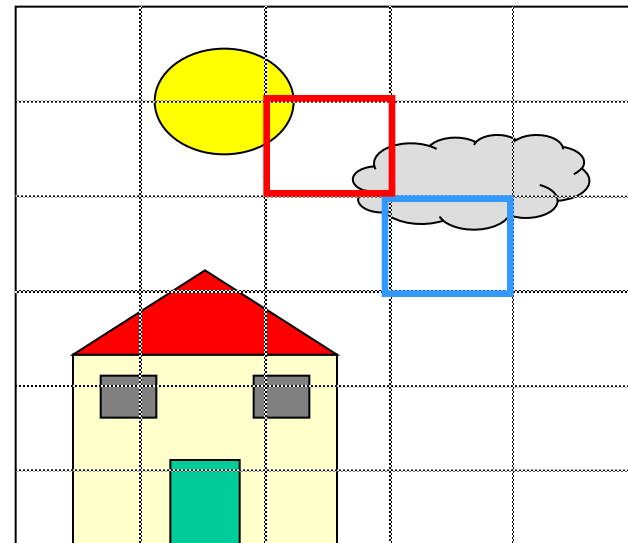


Motion Vectors

- A *motion vector* (MV) describes the offset between the location of the block being coded (in the current frame) and the location of the best-match block in the reference frame
- In this example most blocks can be accurately predicted with MV, with a few exceptions



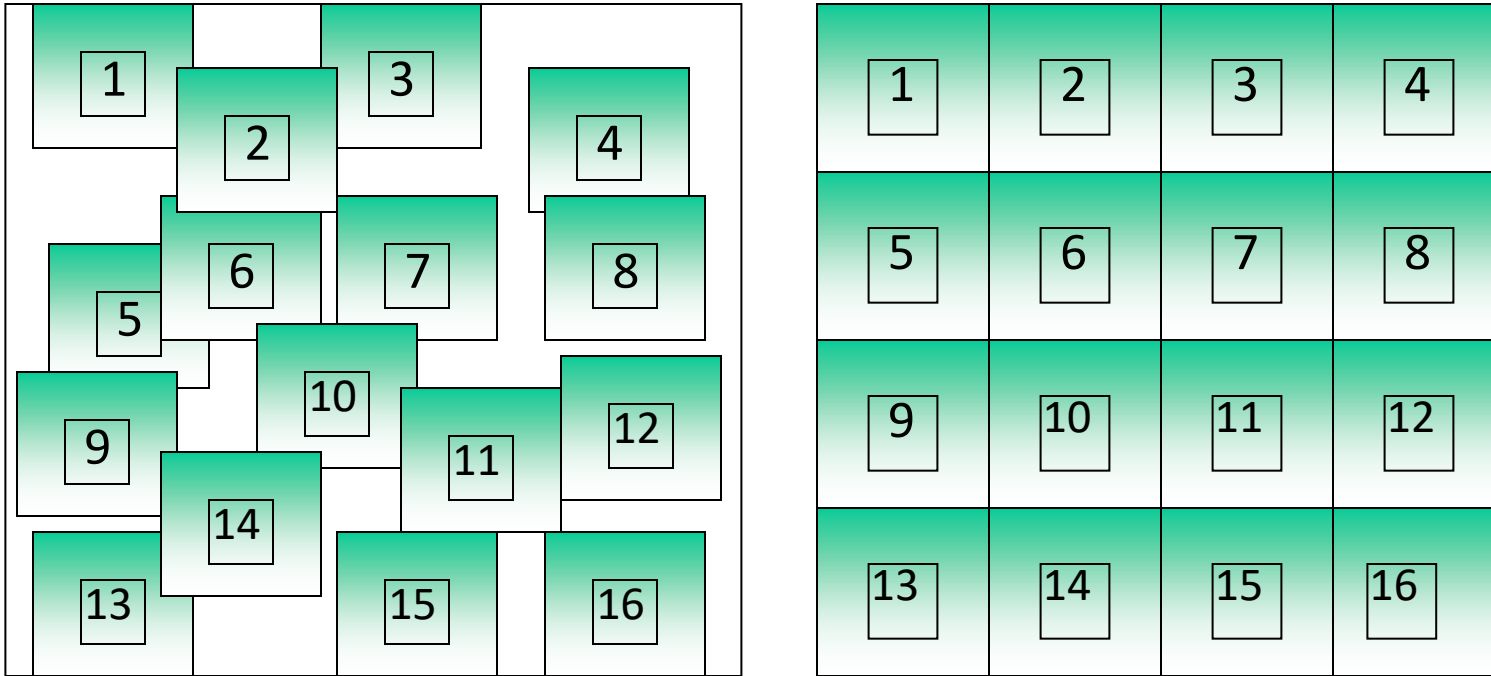
T=1 (reference)



T=2 (current)

Motion Compensation

- The blocks being predicted are on a grid



- The blocks used for prediction can be anywhere

Encoding Difference Frames

- Encoder computes (i) Motion Vectors (MV) and (ii) Motion Compensated Diff (MCD) frame:

$$MCD(n) = F(n) - M(n)$$

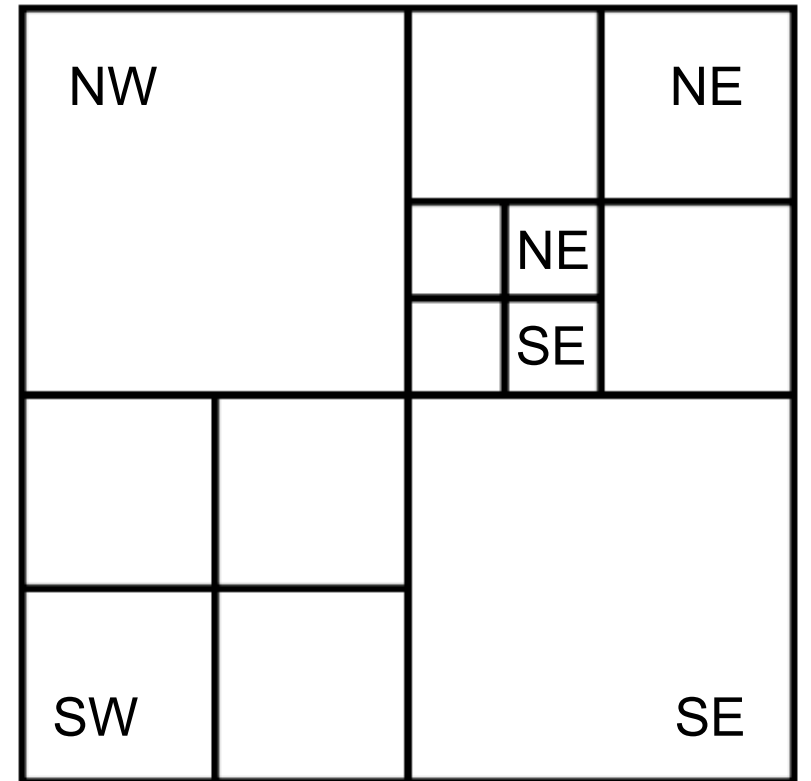
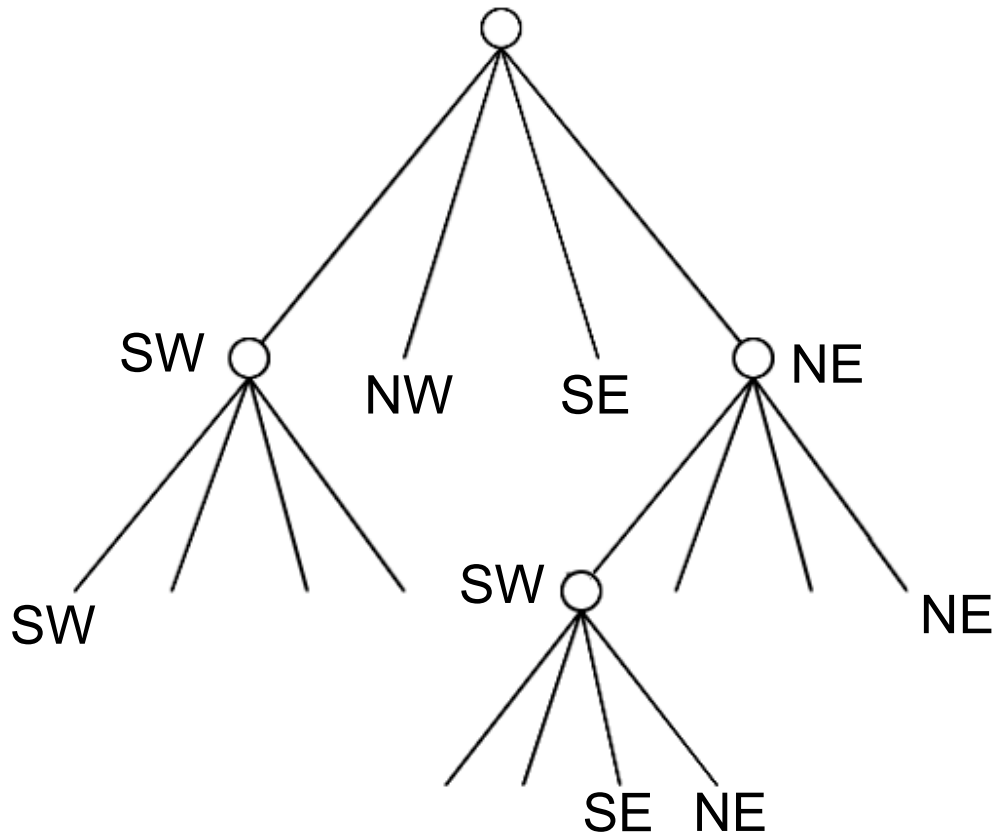
- Encoder encodes MVs and $MCD(n)$
- Decoder receives motion vectors + MCD and can compute the target frame (from previous, known, frame):

$$F(n) = MCD(n) + M(n)$$

- If successive frames are similar, fewer bits are needed to send Motion Vectors + $MCD(n)$

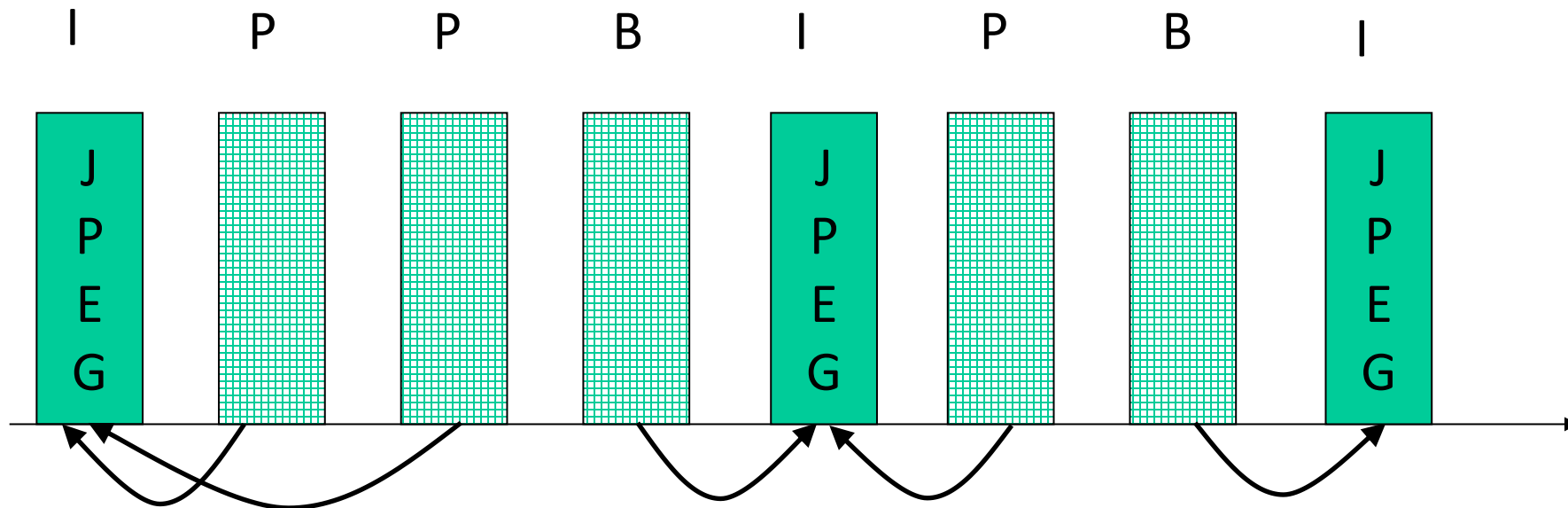
Advanced Video Compression (e.g. H265)

- Adaptive block size with a Quad Tree



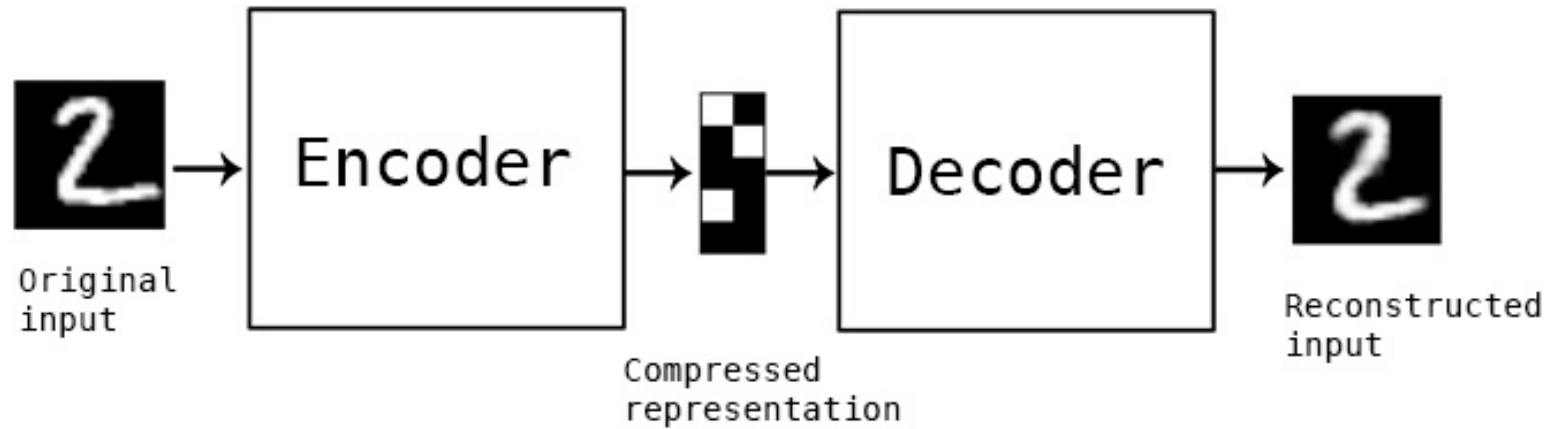
A Coding Example (MPEG)

- Code frames as two types
 - I - frames: Independent JPEG compression
 - P - frames: predicted frames
 - B - frames: bidirectional predicted frames
- An example of a coded sequence:



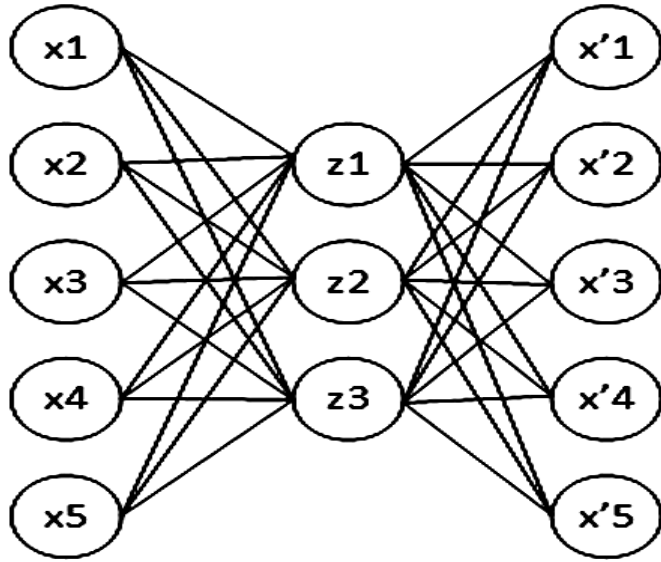
Extra Neural Network

Neural Networks: Autoencoders



- Learn Encoding and Decoding using Neural Networks, where the output should be equal to input.
- No need to create “supervised” training.

Deep Autoencoders

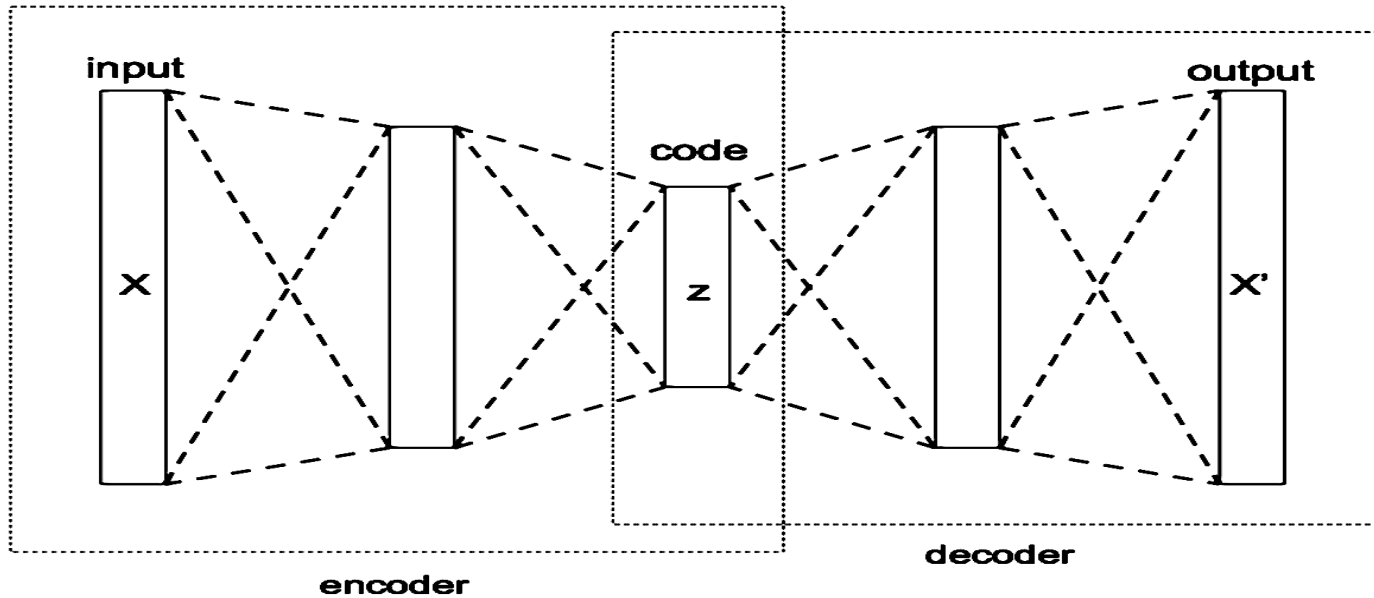


History:

The “Cat” paper – Google, 2012

Autoencoder trained on 10M images
could recognize cats...

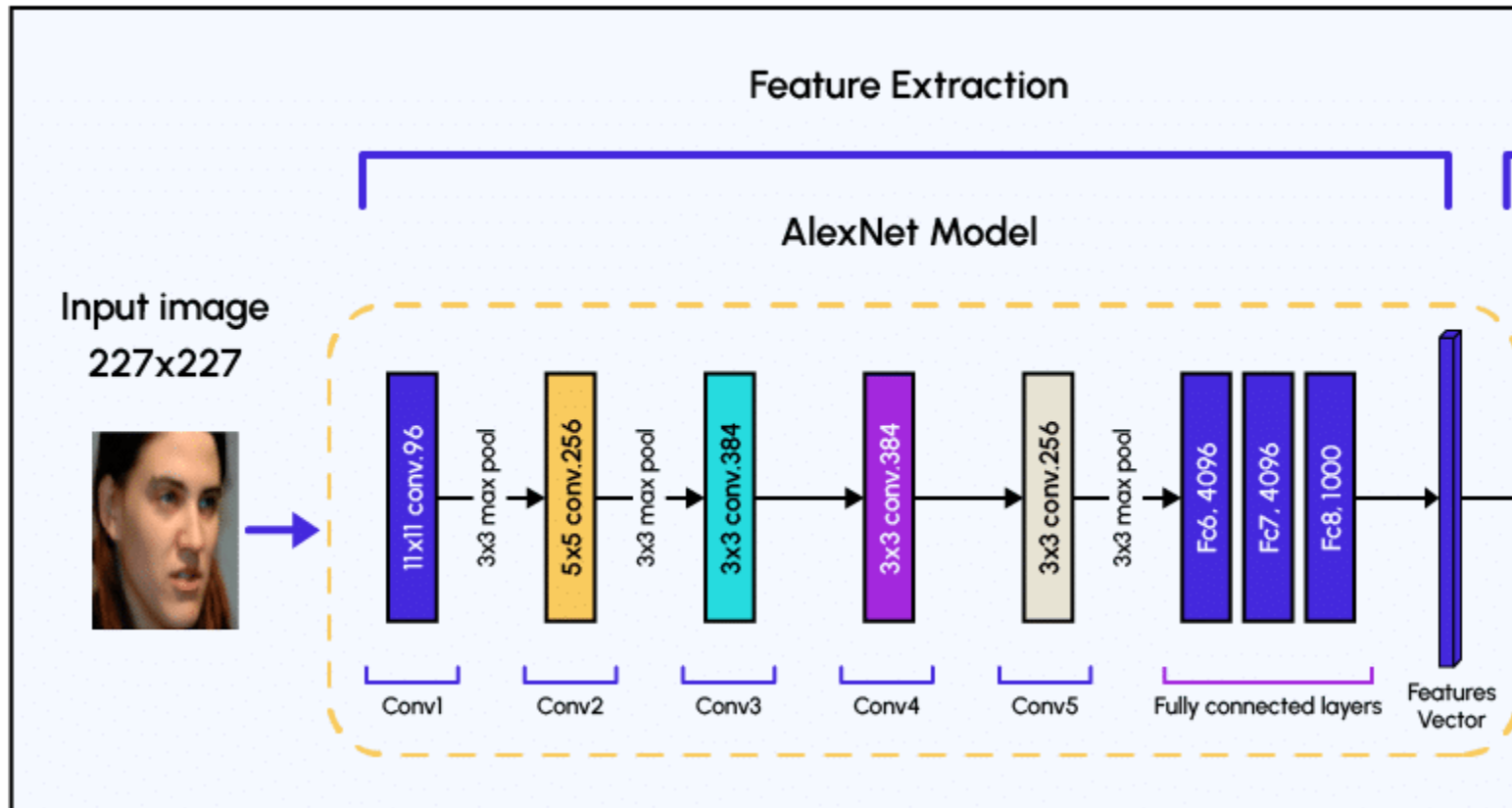
No supervision!



$$J = \sum_{n=1}^N |x(n) - \hat{x}(n)|^2$$

Example: Face Recognition

- Given a face image, train a CNN to give a descriptor vector of length 1024 (Or any other length)

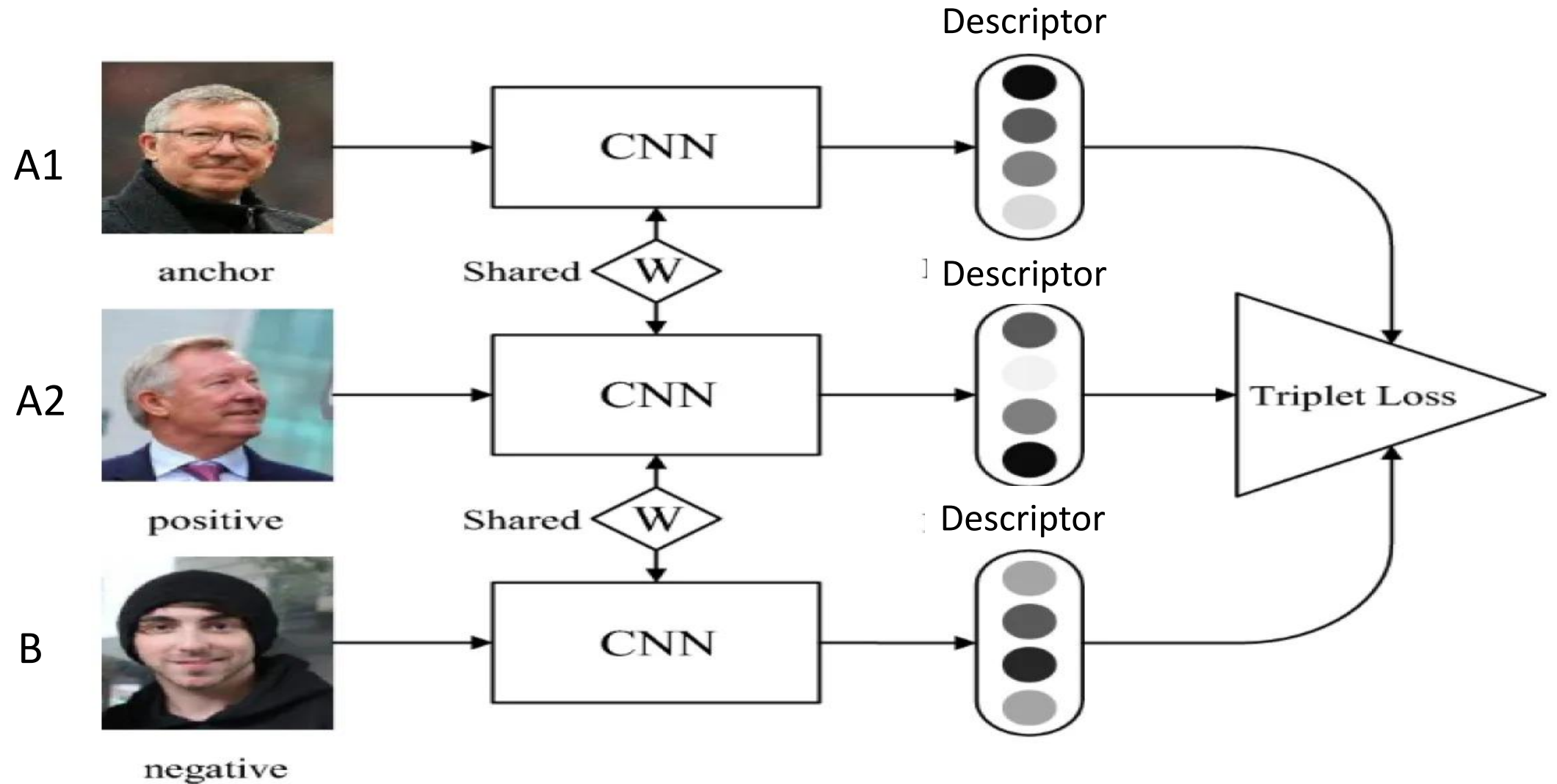


Example: Face Recognition

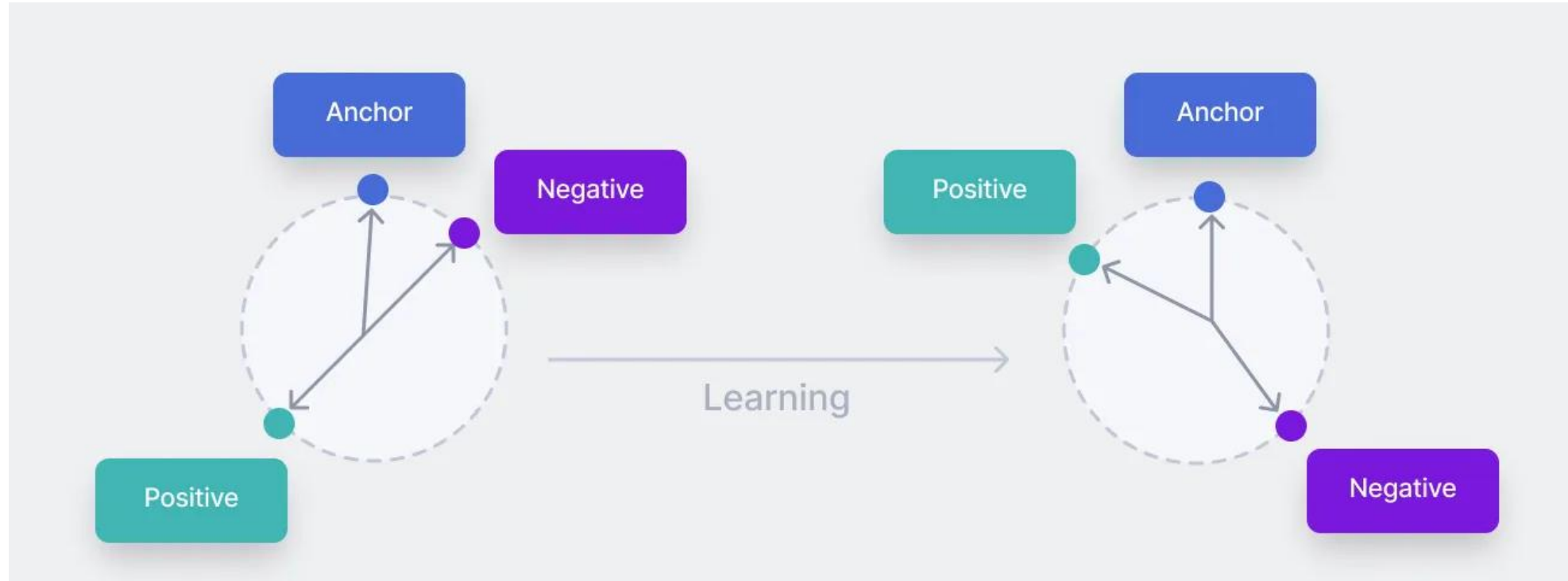
- Train the network such that the descriptor for two different images of **same person** will be closer to each other compared to the descriptors of **different people**.
- Say A1 and A2 are pictures of same person, and B is a picture of another person, maximize the descriptor differences (**Triplet Loss**) & Train over millions of people.

$$\text{Triplet Loss} = \text{Dist}(B, A1) - \text{Dist}(A2, A1)$$

Triplet Loss

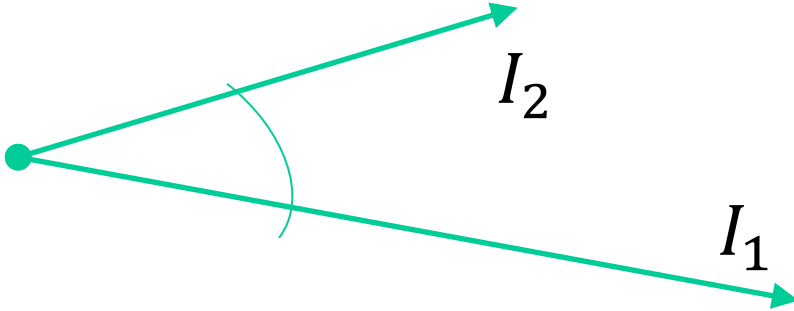


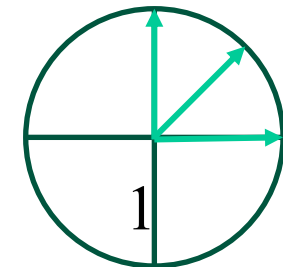
Triplet Loss



- Given a gallery with pictures of known people, and given a query picture, find the person whose descriptor is closest to the query image, and verify it is close.

Similarity Between Vectors

- We have derived from SSD:
$$L^2(I_1, I_2) = \sum_x (I_1(x) - I_2(x))^2$$
- The Inner Product (Correlation):
$$CC(I_1, I_2) = \sum_x I_1(x) \cdot I_2(x)$$
- Normalized CC:
$$NCC(I_1, I_2) = \frac{\sum (I_1(x) - \hat{I}_1) \cdot (I_2(x) - \hat{I}_2)}{\sqrt{\sum (I_1(x) - \hat{I}_1)^2} \sqrt{\sum (I_2(x) - \hat{I}_2)^2}}$$
- **Cosine Similarity:** 
$$\frac{\sum_x (I_1(x) \cdot I_2(x))}{\|I_1(x)\| + \|I_2(x)\|}$$
- Works in any dimension, scale invariant



Cosine Similarity not Shift Invariant

